

四阶龙格-库塔法求解一阶常微分方程组上机报告

周才翔

数试2402 2243711412

2026 年 5 月 19 日

1 四阶龙格-库塔法原理

1.1 引言

四阶龙格-库塔法（Runge-Kutta, R-K4）是基于泰勒展开思想构造的显式单步数值方法。其核心思想是通过计算求解区间内多个点的函数值，对这些值进行加权平均来近似泰勒展开的高阶项，从而在避免直接计算复杂高阶导数的同时，获得较高的计算精度。

对于一阶常微分方程组，其数学形式与单个一阶常微分方程完全一致，仅需将标量函数替换为向量函数。因此，单个方程的四阶龙格-库塔法可以直接推广到方程组的求解，无需修改算法的核心逻辑。

1.2 标准四阶龙格-库塔法公式

1.2.1 单个一阶常微分方程

对于一阶常微分方程初值问题：

$$\begin{cases} y' = f(x, y) \\ y(x_0) = y_0 \end{cases}$$

标准四阶龙格-库塔法的计算公式为：

$$\begin{cases} y_{i+1} = y_i + \frac{1}{6}(K_1 + 2K_2 + 2K_3 + K_4) \\ K_1 = hf(x_i, y_i) \\ K_2 = hf\left(x_i + \frac{h}{2}, y_i + \frac{K_1}{2}\right) \\ K_3 = hf\left(x_i + \frac{h}{2}, y_i + \frac{K_2}{2}\right) \\ K_4 = hf(x_i + h, y_i + K_3) \end{cases}$$

其中 h 为步长, K_1, K_2, K_3, K_4 为四个不同点的函数增量。该方法的局部截断误差为 $O(h^5)$, 整体截断误差为 $O(h^4)$, 因此被称为四阶方法。

1.2.2 一阶常微分方程组（向量形式）

对于 n 维一阶常微分方程组初值问题：

$$\begin{cases} \mathbf{y}'(x) = \mathbf{f}(x, \mathbf{y}(x)), & a \leq x \leq b \\ \mathbf{y}(a) = \mathbf{y}_0 \end{cases}$$

其中 $\mathbf{y}(x) = (y_1(x), y_2(x), \dots, y_n(x))^T$ 为解向量, $\mathbf{f}(x, \mathbf{y}) = (f_1(x, \mathbf{y}), f_2(x, \mathbf{y}), \dots, f_n(x, \mathbf{y}))^T$ 为右端函数向量。其向量形式的四阶龙格-库塔法公式为：

$$\begin{cases} \mathbf{y}_{i+1} = \mathbf{y}_i + \frac{1}{6}(\mathbf{K}_1 + 2\mathbf{K}_2 + 2\mathbf{K}_3 + \mathbf{K}_4) \\ \mathbf{K}_1 = h\mathbf{f}(x_i, \mathbf{y}_i) \\ \mathbf{K}_2 = h\mathbf{f}\left(x_i + \frac{h}{2}, \mathbf{y}_i + \frac{\mathbf{K}_1}{2}\right) \\ \mathbf{K}_3 = h\mathbf{f}\left(x_i + \frac{h}{2}, \mathbf{y}_i + \frac{\mathbf{K}_2}{2}\right) \\ \mathbf{K}_4 = h\mathbf{f}(x_i + h, \mathbf{y}_i + \mathbf{K}_3) \end{cases}$$

其分量形式为：

$$\begin{cases} y_{j,i+1} = y_{j,i} + \frac{1}{6}(K_{j1} + 2K_{j2} + 2K_{j3} + K_{j4}) \\ K_{j1} = hf_j(x_i; y_{1i}, y_{2i}, \dots, y_{ni}) \\ K_{j2} = hf_j\left(x_i + \frac{h}{2}; y_{1i} + \frac{K_{11}}{2}, y_{2i} + \frac{K_{21}}{2}, \dots, y_{ni} + \frac{K_{n1}}{2}\right) \\ K_{j3} = hf_j\left(x_i + \frac{h}{2}; y_{1i} + \frac{K_{12}}{2}, y_{2i} + \frac{K_{22}}{2}, \dots, y_{ni} + \frac{K_{n2}}{2}\right) \\ K_{j4} = hf_j(x_i + h; y_{1i} + K_{13}, y_{2i} + K_{23}, \dots, y_{ni} + K_{n3}) \end{cases} \quad (j = 1, 2, \dots, n)$$

1.3 方法优点

1. 精度高：具有四阶精度，能够满足大多数科学计算和工程问题的精度要求。
2. 步长灵活：每一步的步长可以根据精度要求独立调整，无需全局固定步长。
3. 易于实现：算法逻辑清晰，无需求解非线性方程组，编程实现简单。

2 Python编程实现

```
import math
```

```
def runge_kutta_4_system(f, y0, a, b, h):

    n = len(y0)
    n_steps = int(round((b - a) / h))
    x_list = [a + i * h for i in range(n_steps + 1)]
    y_list = [y0.copy()]

    current_y = y0.copy()
    for i in range(n_steps):
        x = x_list[i]
        k1 = [h * fi for fi in f(x, current_y)]
        y_k2 = [current_y[j] + k1[j] / 2 for j in range(n)]
        k2 = [h * fi for fi in f(x + h/2, y_k2)]
        y_k3 = [current_y[j] + k2[j] / 2 for j in range(n)]
        k3 = [h * fi for fi in f(x + h/2, y_k3)]
        y_k4 = [current_y[j] + k3[j] for j in range(n)]
        k4 = [h * fi for fi in f(x + h, y_k4)]
        next_y = [
            current_y[j] + (k1[j] + 2*k2[j] + 2*k3[j] + k4[j]) / 6
            for j in range(n)
        ]

        y_list.append(next_y)
        current_y = next_y

    return x_list, y_list

def main():
    print("四阶龙格-库塔法求解一阶常微分方程组")

    n = int(input("请输入方程组的维数n: "))
    equations = []
    print(f"\n请输入{n}个方程的右端函数表达式")
    print("示例: y[1]、math.exp(2*x)*math.sin(x) - 2*y[0] + 2*y[1]")
    for i in range(n):
        expr = input(f"方程{i+1} (y[{i}]' = ): ").strip()
```

```

        equations.append(expr)

def system_f(x, y):
    result = []
    for expr in equations:
        result.append(eval(expr, {"math": math}, {"x": x, "y": y}))
    return result

y0_input = input("\n请输入初始条件y(a), 用逗号分隔 (例如: -0.4,-0.6):")
y0_input.strip()
y0 = [float(num.strip()) for num in y0_input.split(',')]
a = float(input("\n请输入求解区间左端点a: ").strip())
b = float(input("请输入求解区间右端点b: ").strip())
h = float(input("请输入步长h: ").strip())

x_list, y_list = runge_kutta_4_system(system_f, y0, a, b, h)

print(f"{'x':^10} | {'y[0]':^18} | {'y[1]':^18} | {'...':^18}")
for i in range(len(x_list)):
    x = x_list[i]
    y = y_list[i]
    y_str = " | ".join([f"{yi:^18.6f}" for yi in y])
    print(f"{x:^10.4f} | {y_str}")

if __name__ == "__main__":
    main()

```

2.1 代码使用说明

2.1.1 输入要求

1. **方程组维数 n** : 正整数, 表示一阶方程组的方程个数。
2. **方程右端函数**: 每个方程的表达式, 使用“ x ”表示自变量, “ $y[0], y[1], \dots, y[n-1]$ ”表示因变量分量, 可使用math模块中的所有数学函数。
3. **初始条件**: 用逗号分隔的 n 个数字, 对应 $\mathbf{y}(a)$ 的各个分量。
4. **求解区间**: 左端点 a 和右端点 b , 要求 $b > a$ 。
5. **步长 h** : 正数, 且能整除区间长度 $b - a$ 。

2.1.2 输出结果

程序将以表格形式输出所有节点的 x 值和对应的 y 向量各个分量值，保留6位小数。

3 实例计算与结果分析

3.1 算例描述（教材9.3.2节例9.3.1）

求解二阶常微分方程初值问题：

$$\begin{cases} y'' - 2y' + 2y = e^{2x} \sin x, & 0 \leq x \leq 1 \\ y(0) = -0.4, & y'(0) = -0.6 \end{cases}$$

将其转化为一阶常微分方程组：令 $y_1 = y$, $y_2 = y'$, 则：

$$\begin{cases} y_1' = y_2 \\ y_2' = e^{2x} \sin x - 2y_1 + 2y_2 \end{cases}$$

初始条件： $y_1(0) = -0.4$, $y_2(0) = -0.6$, 求解区间为 $[0, 1]$, 步长 $h = 0.1$ 。

该问题的解析解为：

$$y(x) = 0.2e^{2x}(\sin x - 2\cos x)$$

其导数为：

$$y'(x) = 0.2e^{2x}(4\sin x - 3\cos x)$$

3.2 输入参数

运行上述代码后，按照以下提示依次输入：

四阶龙格-库塔法求解一阶常微分方程组

请输入方程组的维数 n : 2

请输入2个方程的右端函数表达式

示例: $y[1]$ 、 $\text{math.exp}(2*x)*\text{math.sin}(x) - 2*y[0] + 2*y[1]$

方程1 ($y[0]' =$): $y[1]$

方程2 ($y[1]' =$): $\text{math.exp}(2*x)*\text{math.sin}(x) - 2*y[0] + 2*y[1]$

请输入初始条件 $y(a)$ ，用逗号分隔（例如: $-0.4, -0.6$): $-0.4, -0.6$

请输入求解区间左端点 a : 0

请输入求解区间右端点 b : 1

请输入步长 h : 0.1

3.3 计算结果

程序运行后输出以下结果：

x		y[0]		y[1]		...
0.0000		-0.400000		-0.600000		
0.1000		-0.461733		-0.631631		
0.2000		-0.525560		-0.640149		
0.3000		-0.588601		-0.613664		
0.4000		-0.646612		-0.536582		
0.5000		-0.693567		-0.388738		
0.6000		-0.721152		-0.144381		
0.7000		-0.718153		0.228997		
0.8000		-0.669711		0.771992		
0.9000		-0.556443		1.534781		
1.0000		-0.353399		2.578766		

比较解析解可以看出，数值解与解析解的绝对误差均在 10^{-6} 量级，充分验证了代码的正确性和四阶龙格-库塔法的高精度特性。

4 总结

本报告简述了四阶龙格-库塔法的核心原理与数学公式，实现了一阶常微分方程组求解程序，并通过教材9.3.2节的二阶常微分方程初值问题进行了验证。结果表明，四阶龙格 - 库塔法的数值解与解析解的绝对误差控制在 10^{-6} 量级，兼具较高的计算精度与适中的计算量。四阶龙格 - 库塔法兼顾了精度、稳定性与易用性，是求解常微分方程初值问题的经典方法，在科学计算、工程仿真等领域具有广泛的应用价值。